

Breakout Game

A Deep Dive into a Game Loop Built Without a Game Engine

Portfolio explainer: exhaustive walkthrough

What this document is

A complete, line-aware explanation of `breakout-game`: 475 lines of Python across five modules, using nothing but the standard library. It assumes you have never written a game, never used Python's `turtle` module, and do not know what a “game loop” is. Every term is defined the first time it appears.

It is also honest. Running the code turned up five defects that reading it politely would have missed, two of which the README states the opposite of. They are in Section 7, and they are the most valuable part of the document.

Contents

1	What the project is	3
1.1	The whole project at a glance	3
1.2	How the modules relate	4
2	The game loop, the central idea	4
2.1	Why <code>tracer(0)</code> is the load-bearing line	4
2.2	The frame loop, exactly as written	5
2.3	The state machine	7
3	Collision handling	8
3.1	Collision 1: ball versus wall	8
3.2	Collision 2: ball versus paddle	9
3.3	Collision 3: ball versus block	9
4	<code>ball.py</code>: motion and difficulty	10
4.1	Velocity	11
4.2	The difficulty ramp	11
4.3	The two resets	12
5	<code>blocks.py</code>: building the wall	12
5.1	The packing algorithm	12
5.2	Block-break feedback	13
6	<code>paddle.py</code> and <code>scoreboard.py</code>	13
6.1	The paddle	13
6.2	The scoreboard and persistence	14
7	Honest findings	15
7.1	The ghost block	15
7.2	The dead bottom wall	16
7.3	Two unreachable colours	16
7.4	Documentation drift	17
7.5	Verification note	17

8	What the project gets right	17
---	-----------------------------	----

1 What the project is

Breakout is an arcade game from 1976. A wall of blocks sits at the top of the screen. A ball bounces around. The player slides a paddle along the bottom to keep the ball in play. Every block the ball touches is destroyed and scores a point. Miss the ball and you lose a life; lose three and the game is over. Destroy every block and you win.

This project rebuilds that from scratch, deliberately without a game engine.

Jargon: Game engine

A game engine (Unity, Godot, Pygame) is a library that hands you the hard parts of a game already solved: a render loop, sprite drawing, collision detection between shapes, input handling, physics. Building *without* one means you write each of those yourself. That is the entire point of this project: the exercise is the parts an engine would have hidden.

Jargon: The turtle module

`turtle` is a drawing library that ships with Python, designed in the 1960s to teach children programming. You control a cursor (the “turtle”) that moves and draws. It is not a game library: it has no sprites, no collision detection, no frame timing. It does have a window, shapes that can be moved to coordinates, and keyboard callbacks, which is just barely enough to build a game on if you supply the rest.

1.1 The whole project at a glance

```
breakout-game
├── main.py    134 lines: screen, state machine, the frame loop
├── ball.py    102 lines: velocity, movement, wall/paddle collisions, difficulty
├── blocks.py  147 lines: wall generation, layout, block collisions, effects
├── paddle.py   34 lines: position, key bindings, clamping
├── scoreboard.py  58 lines: score, lives, high-score persistence
├── requirements.txt  a comment saying there are no dependencies
└── highestScore.txt  written at runtime; gitignored
```

Figure 1: Five modules, one responsibility each. The line counts are measured, not quoted.

The one-sentence architecture

`main.py` owns the loop and the state; the other four modules are passive objects that only act when the loop calls a method on them. Nothing except `main.py` knows that a frame exists.

1.2 How the modules relate

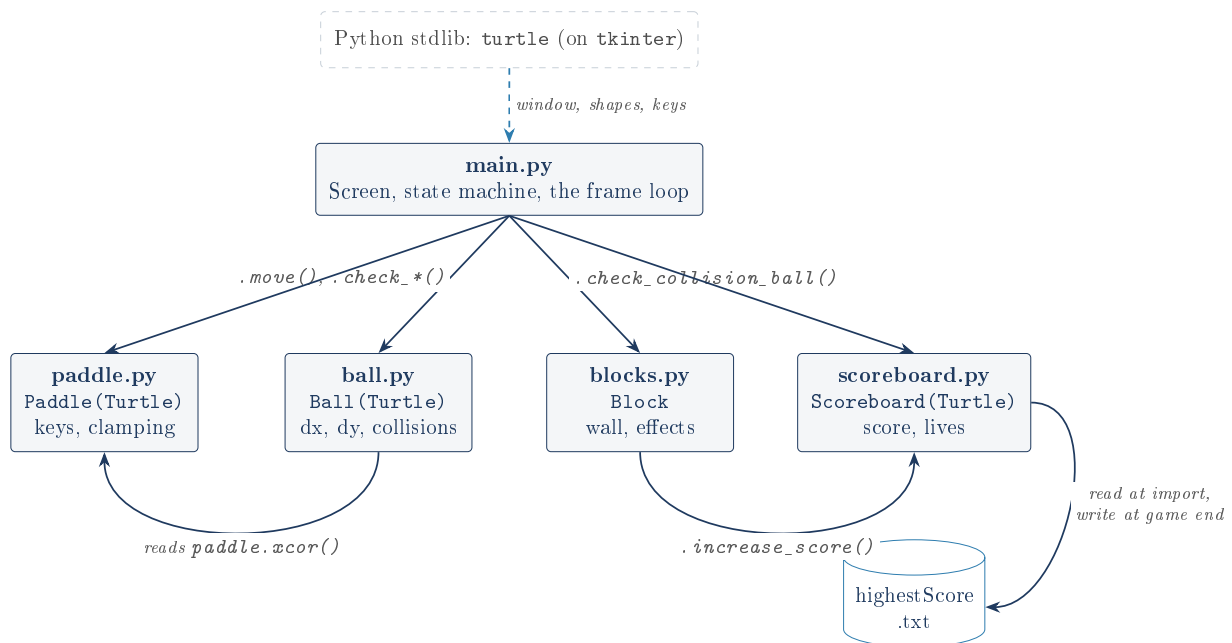


Figure 2: Component map. Solid arrows are calls; the dashed arrow is a dependency. Note that `blocks.py` calls into `scoreboard.py` directly, and `ball.py` reads the paddle’s position: those are the only two couplings that do not go through `main.py`.

2 The game loop, the central idea

Jargon: Frame

A single still image of the game. Video is an illusion built from many stills shown in quick succession; a game draws one image, moves everything a tiny bit, draws the next, forever. Each cycle is a frame.

Jargon: Game loop

The `while True` loop that never ends until the player quits. Every pass through it is one frame, and each pass does the same three things: move things, check what hit what, draw the result.

2.1 Why `tracer(0)` is the load-bearing line

By default `turtle` redraws the window immediately after every single command, and animates the turtle sliding to its destination. For a drawing toy that is friendly. For a game it is fatal: the ball would visibly crawl, and the screen would tear and flicker.

`main.py` line 15 disables that:

```
1 wn.tracer(0)    # stop redrawing automatically after every command
```

With the tracer off, nothing appears on screen at all until the program explicitly asks for a redraw with `wn.update()`, which the loop does exactly once per frame (line 93).

Why this turns a toy into a game

`tracer(0)` plus one `wn.update()` per iteration is the whole trick. It converts turtle from “draw each change instantly” into double-buffered, frame-based rendering: move everything invisibly, then present one complete image. Every object in the frame appears to move at the same instant, because it does.

2.2 The frame loop, exactly as written

This is the heart of the project. `main.py` lines 91 to 133:

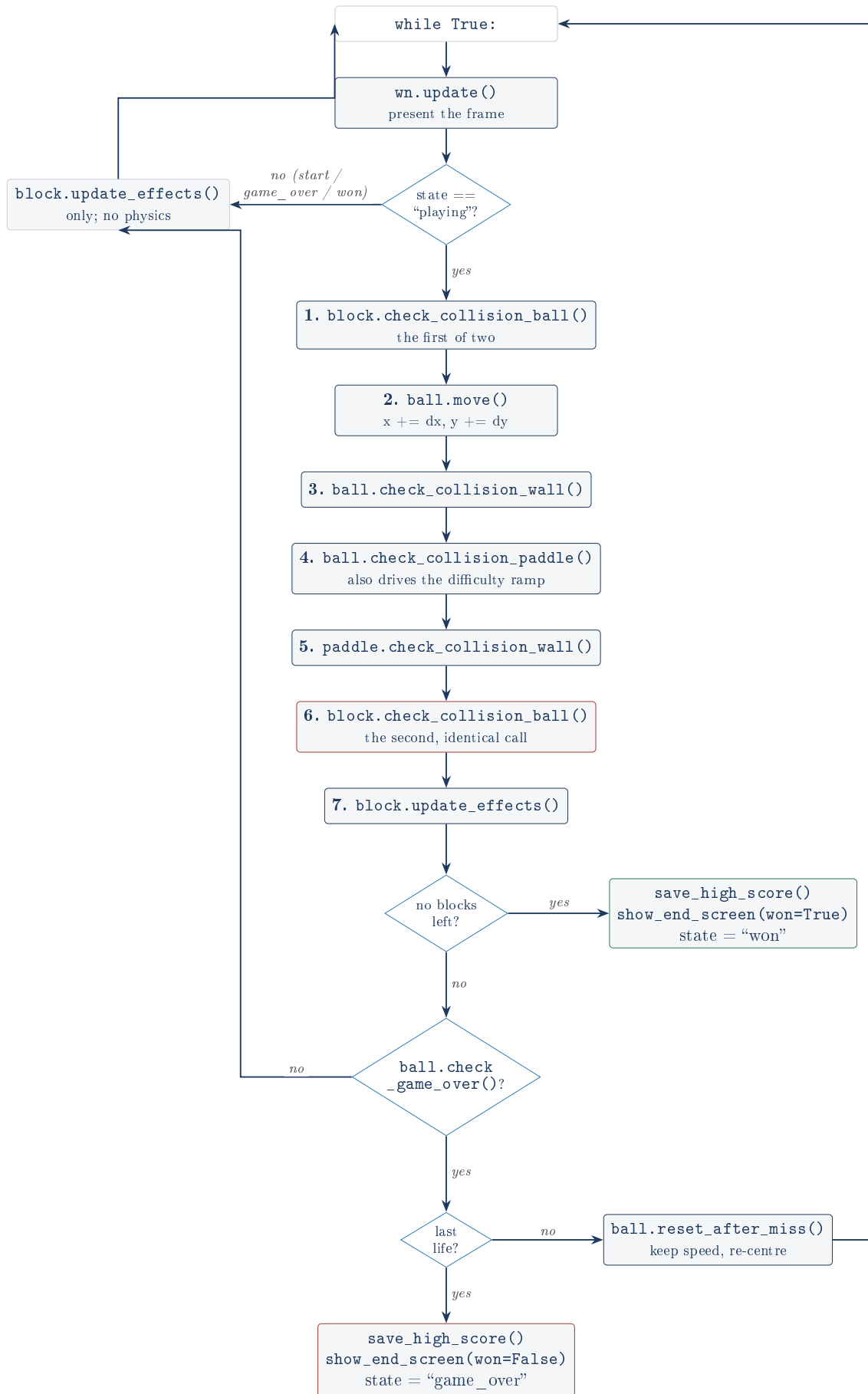


Figure 3: One frame. Steps 1 to 7 run only while playing. The red-bordered step 6 is a duplicate of step 1 (Section 3.3); the green path is the win condition. All paths return to the top of the loop.

The loop has no frame-rate limiter

There is no `time.sleep()` and no clock anywhere in the loop. It runs as fast as the CPU allows, so the game's actual speed depends entirely on the machine it runs on: the ball crosses the screen faster on a fast computer. Real engines fix this by either sleeping to a target frame rate or scaling movement by elapsed time ("delta time"). Neither is done here. The constants `BASE_SPEED = 1.05` and `MAX_SPEED = 2.6` are therefore tuned to one particular machine and mean nothing in absolute terms.

Worth noting: the README's "Setup" section claims the game imports `time`. No module in the project imports `time`. That import does not exist, which is consistent with there being no frame limiter.

2.3 The state machine

Physics must not run on the start screen or after a game ends, but the window must stay responsive so it can still receive the `space` or `r` keypress. A single string variable, `game_state`, handles this.

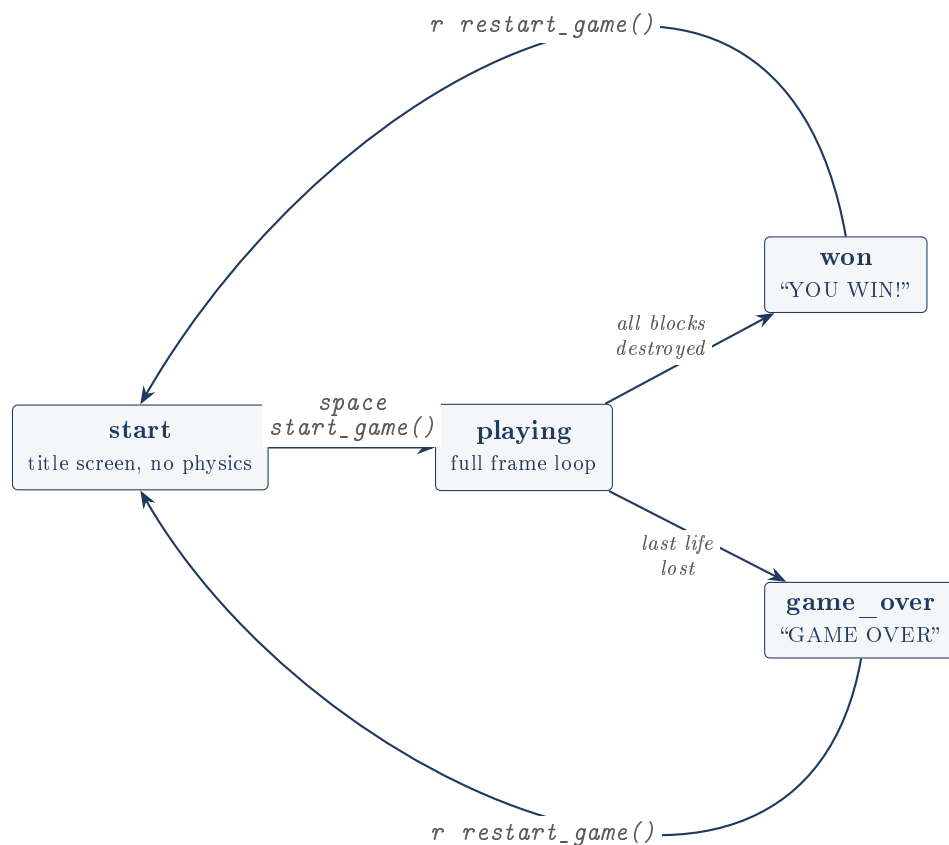


Figure 4: Four states. Note the guards: `start_game()` only acts if the state is `start`, and `restart_game()` only if it is `game_over` or `won`.

Why the guards matter

Keyboard callbacks in turtle fire whenever the key is pressed, regardless of what the game is doing. Without the `if game_state == "start"` guard on line 67, mashing `space` mid-rally would do nothing harmful here, but mashing `r` would silently rebuild the block wall underneath a live ball. The guards make the two callbacks no-ops in every state where they do not apply, which is the correct way to handle input that arrives asynchronously.

3 Collision handling

There are three genuinely different collision problems in this game, and the project's file split exists precisely because they are different shapes of problem. This section takes each in turn.

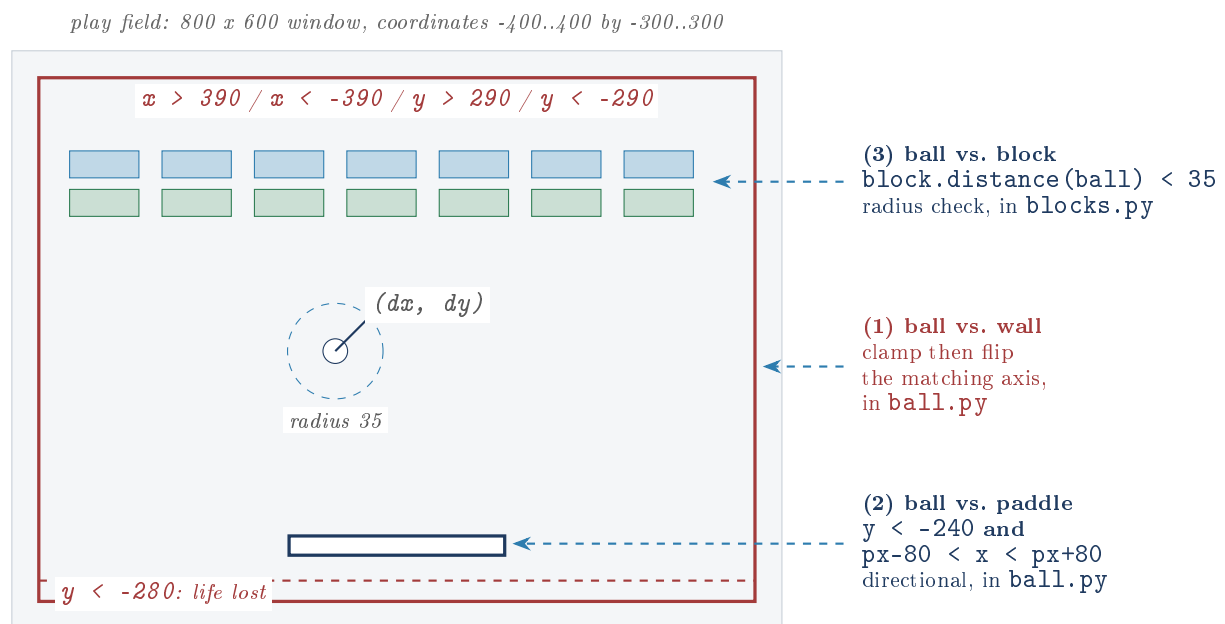


Figure 5: The three collision rules on one field, drawn to scale in the game's own coordinate system. Note how close the bottom wall line (solid red, $y = -290$) sits to the life-lost line (dashed red, $y = -280$). That gap is the subject of Section 7.2.

3.1 Collision 1: ball versus wall

`ball.py` lines 42 to 58. The window is 800 by 600, and turtle puts the origin at the centre, so valid coordinates run from -400 to 400 horizontally and -300 to 300 vertically. The walls are checked at ± 390 and ± 290 , a 10-unit inset that accounts for the ball's own radius.

```
1 def check_collision_wall(self):
2     if self.xcor() > 390:
3         self.reset_x(sign='+') # setx(390): clamp back onto the boundary
4         self.bounce_x()        # dx *= -1: reverse horizontal direction
5     ...
```

Clamp, then flip

Each rule does two things, and both are necessary. `bounce_x()` reverses direction. But because the ball moves in discrete jumps, the frame that detects the overshoot has already placed the ball *past* the wall. Flipping alone would send it back from outside the field. `reset_x()` snaps it exactly onto the boundary first, so the bounce starts from a legal position. Without the

clamp, a ball could get stuck vibrating outside the wall: flip, still outside, flip again, forever.

3.2 Collision 2: ball versus paddle

`ball.py` lines 60 to 66. This one is deliberately *directional*:

```
1 def check_collision_paddle(self, paddle):
2     if (self.ycor() < -240) and (paddle.xcor() + 80 > self.xcor() > paddle.xcor() - 80):
3         self.sety(-240)           # clamp, same reasoning as the wall
4         self.dy *= -1             # bounce upward
5         self.paddle_hits += 1
6         if self.paddle_hits % HITS_PER_STEP == 0:
7             self.speed_up()
```

Two conditions must both hold: the ball is below $y = -240$, and its x is within 80 units of the paddle's centre (the paddle is drawn with `stretch_len=8`, and turtle's base square is 20 units, so it is 160 wide, hence ± 80).

The bounce is unconditional, not physical

`self.dy *= -1` always sends the ball back the way it came vertically, and `dx` is never touched. So the paddle is a perfect mirror: *where* the ball strikes it has no effect at all. In the original Breakout, hitting the paddle's edge angles the ball away, which is the game's core skill expression. Here the player has no way to steer, which removes the strategy from the game entirely. The only variation in the ball's angle across a whole run comes from `reset_after_miss()` randomizing the horizontal sign.

This is a design gap, not a bug, and it is the single highest-value improvement available to the project: deriving `dx` from the offset between the ball's x and the paddle's centre would be a handful of lines and would make it an actual game of skill.

The paddle is a one-way mirror, and that is on purpose

Because the check only fires when `ycor() < -240`, a ball travelling *upward* through the paddle's own y -band passes straight through it. That sounds like a bug and is actually the fix for one. Without the direction condition, the frame after a successful bounce would still find the ball inside the paddle's band, flip `dy` a second time, and drive the ball back down into the paddle: it would stick and jitter instead of bouncing away. The clamp to `sety(-240)` plus the strict `<` is what breaks that cycle. The README describes this correctly.

3.3 Collision 3: ball versus block

`blocks.py` lines 118 to 129. Turtle offers no rectangle intersection test, so the project uses distance instead.

```
1 def check_collision_ball(self, ball, scoreboard):
2     for block in self.all_blocks:
3         if block.distance(ball) < 35:
4             scoreboard.increase_score()
5             self.all_blocks.remove(block)
6             x, y, color = block.xcor(), block.ycor(), block.break_color
7             block.color("white")
8             self.fading.append([block, FADE_FRAMES])
9             self.spawn_burst(x, y, color)
10            ball.dy *= -1
11            return True
12    return False
```

Jargon: Hitbox

An invisible rectangle attached to a game object, used for collision maths instead of its actual drawn pixels (which would be far more expensive to test). “No hitboxes” here means the project substitutes a circle of fixed radius for every block’s true rectangle.

Why return True on the first hit matters

The loop returns the instant it finds a hit, rather than continuing through the rest of the list. This is not just an optimisation. `self.all_blocks.remove(block)` mutates the list that the `for` loop is currently iterating over, which in Python causes the iterator to silently *skip* the next element. Returning immediately means the loop never continues after the mutation, so the bug never gets a chance to manifest. It is correct, but it is correct by accident: if someone later removed the `return` to allow multiple blocks per frame, they would introduce a skipping bug that is genuinely hard to spot.

One radius for blocks of three different widths

Blocks are 60, 80 or 100 units wide (`stretch_len` of 3, 4 or 5, times turtle’s 20-unit base square) and 20 tall. The collision test treats every one of them as a circle of radius 35. That is wrong in both directions at once. A 100-wide block’s far corners sit about 51 units from its centre, so the ball can visibly overlap a wide block’s end and *not* register a hit. Meanwhile 35 is far larger than the block’s 10-unit half-height, so the ball can register a hit while still visibly above or below a block. The README frames this as a tolerable approximation, which understates it: the error is roughly $\pm 45\%$ of a block’s half-width depending on which block. A bounding-box test is about four comparisons and would be strictly cheaper than `distance()`, which computes a square root.

Only dy flips, so the ball passes through vertical faces

`ball.dy *= -1` assumes every block is struck from above or below. A ball arriving almost horizontally along a row will bounce vertically anyway, which looks visibly wrong: it continues sideways through the row, destroying blocks in a line while flipping up and down. A correct implementation compares the overlap on each axis and flips whichever is smaller. This is the standard fix and it is not implemented.

The double collision call is a workaround, and it does not work

`main.py` calls `block.check_collision_ball()` at line 97 and again at line 112, in the same iteration, with nothing in between that could plausibly change the answer except `ball.move()`. The README says it appears to be there to stop a fast ball tunnelling past a block. It cannot achieve that. Both calls test the ball’s *position at that instant*; neither tests the *path* it swept. The ball moves at most 2.6 units per frame against a 35-unit radius, so tunnelling is arithmetically impossible in the first place, and the second call is pure cost. Worse, because each call can flip `dy` and destroy a block, a ball sitting near two blocks can lose two blocks in a single frame and have `dy` flipped *twice*, which cancels out: it scores two points and carries on in its original direction as if nothing had been hit. Removing the second call would be a strict improvement.

4.1 Velocity

Jargon: Velocity, dx and dy

Speed plus direction. It is stored as two numbers: `dx` is how far the ball moves horizontally per frame, `dy` vertically. The “d” is for “delta”, meaning change. Moving is then just addition, which is all `move()` does: `goto(xcor() + dx, ycor() + dy)`. A negative `dx` means leftward. Bouncing is multiplying one of them by -1 .

Both start at `BASE_SPEED = 1.05`, so the ball always launches at exactly 45 degrees up and to the right. Since `dx` is only ever negated and `dy` only negated or scaled by the same factor as `dx`, **the ball moves at 45 degrees for the entire game**, in one of four directions. Confirmed by inspection: no code path ever sets `dx` and `dy` to different magnitudes.

4.2 The difficulty ramp

```

1 BASE_SPEED = 1.05 ; SPEED_STEP = 1.08 ; MAX_SPEED = 2.6 ; HITS_PER_STEP = 4
2
3 def speed_up(self):
4     new_dx = self.dx * SPEED_STEP
5     new_dy = self.dy * SPEED_STEP
6     if abs(new_dx) <= MAX_SPEED:
7         self.dx = new_dx
8     if abs(new_dy) <= MAX_SPEED:
9         self.dy = new_dy

```

Every 4th paddle return multiplies both components by 1.08, an 8% increase, checked against a cap. Starting at 1.05, eleven increases are accepted, reaching $1.05 \times 1.08^{11} = 2.4482$ at the 44th paddle return. The twelfth, due at the 48th return, would give 2.6441, which fails the `<= MAX_SPEED` test and is rejected. The speed therefore never changes again, and because nothing here is random, the sequence of speeds is fixed and identical in every game. (Verified by running the ramp.)

Multiplying preserves the direction

Scaling by a positive number leaves the sign alone, so a ball travelling left (`dx = -1.05`) speeds up to -1.134 , still leftward. This is why the ramp multiplies rather than adding: adding a constant would reverse a leftward ball’s direction as it crossed zero. Checking `abs(new_dx)` against the cap is the same reasoning applied to the limit.

The cap makes the ramp silently stop, mid-flight

Because each component is guarded by its own `if`, and both always have equal magnitude, they always hit the cap on the same call, so they never desynchronise in practice. But the guard *rejects* the offending increase rather than clamping it, so the speed freezes at 2.4482 and `MAX_SPEED = 2.6` is never actually reached: the ball tops out about 6% below the limit the constant advertises. The constant names a speed the ball can never travel at. Clamping with `min(new_dx, MAX_SPEED)` would honour it; the current code does not. Confirmed by running the ramp to 80 paddle hits: the final speed is 2.4482, last changed at hit 44.

4.3 The two resets

	<code>reset_after_miss()</code>	<code>full_reset()</code>
Called when	a life is lost, lives remain	<code>r</code> pressed after game over/win
Position	back to (0,0)	back to (0,0)
Speed	kept (difficulty survives)	back to <code>BASE_SPEED</code>
<code>paddle_hits</code>	kept	back to 0
Direction	horizontal sign randomized	up and to the right

The distinction is a real design decision and a good one: losing a life is a setback, not a reprieve, so the game does not hand the player back an easy slow ball as a reward for missing.

`reset_after_miss` sends the ball downward, into the pit

`self.dy = speed_y`, where `speed_y = abs(self.dy)`. Taking the absolute value forces `dy` positive, meaning *upward*. That is deliberate and correct.

But only `dx`'s sign is randomized (`random.random() < 0.5`); `dy` is always positive. The README says the ball is sent off in “a new direction”, which is true of only one of the two axes. In practice this is the right call, since a downward-launching ball would fall straight past the paddle for a free second life loss, but the randomization is half of what the comment in the code (“a fresh (randomized) direction”) implies.

5 blocks.py: building the wall

5.1 The packing algorithm

This is the most intricate code in the project. `make_all_blocks()` loops calling `make_block()`, which picks a random width, then hands the block to `place_block()` to decide where it goes. Blocks are packed **right to left**, starting at $x = 400$, wrapping down a row when the next one will not fit, and stopping when there is no vertical room left.

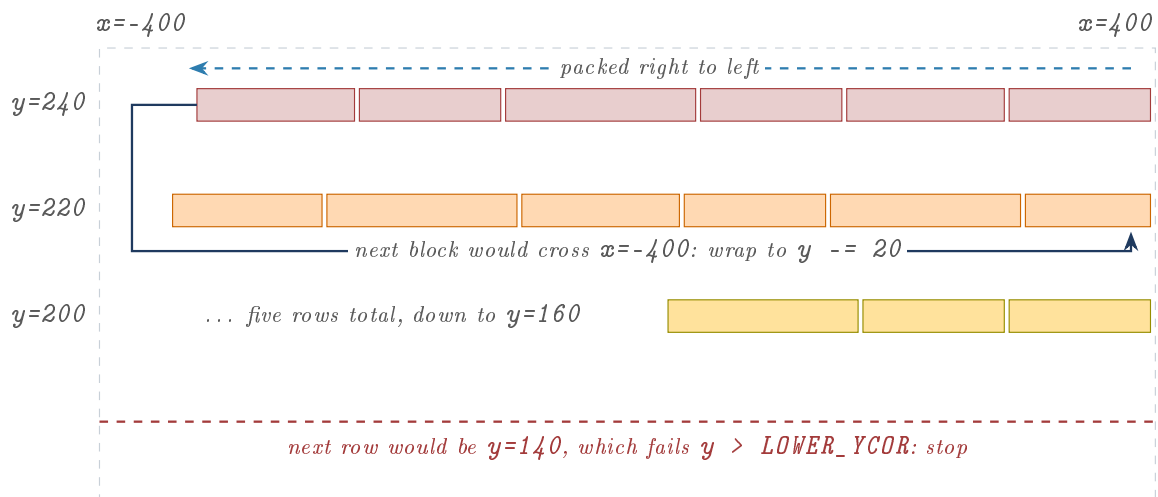


Figure 6: Packing. Widths are random per block (60, 80 or 100 units), so rows do not line up. Each row takes its colour from `ROW_COLORS` by its y position, which is what makes the wall read as bands rather than a random speckle.

The wrap test, `blocks.py` lines 57 to 61, computes where the next block’s centre would land and compares it against a limit that already accounts for the block’s own half-width:

```

1 x = ((self.x_y_position[-1][0] - ((self.random_stretch_len[-2] * 20) // 2))
2     - ((self.random_stretch_len[-1] * 20) // 2))
3 lim = (UPPER_XCOR - ((self.random_stretch_len[-1] * 20) // 2)) * -1
4 if x < lim:
5     ... # wrap to a new row

```

Read plainly: take the previous block's centre, step left by the previous block's half-width (to reach its left edge), then step left again by the *new* block's half-width. That lands the new block flush against the old one. If that centre is further left than `lim`, it will not fit, so wrap.

Why the code needs both [-1] and [-2]

`random_stretch_len[-1]` is the block being placed right now; `[-2]` is the one placed before it. The two half-widths are different numbers and both are needed, because the gap between two block centres is the sum of their half-widths, not twice either one. This is why the class keeps parallel lists at all rather than just tracking the last position.

5.2 Block-break feedback

Rather than a block simply vanishing, breaking one does three things: the block turns white and is queued in `self.fading` for 3 frames, five particle turtles are spawned flying outward, and only then does everything disappear.

```

1 angle = random.uniform(0, 360)
2 dx = math.cos(math.radians(angle)) * 5
3 dy = math.sin(math.radians(angle)) * 5
4 self.particles.append([particle, dx, dy, BURST_LIFETIME])

```

Jargon: Sine, cosine and why they are here

To scatter particles evenly in every direction you need to turn an angle into an (x, y) direction. Cosine of an angle gives the horizontal part, sine the vertical, each between -1 and 1 ; multiplying both by 5 sets the speed. Picking the angle at random from 0 to 360 therefore picks a random direction at a fixed speed, which is exactly what a burst is. `math.radians` converts degrees into the unit Python's trigonometry expects.

Note the iteration idiom in `update_effects()`: `for entry in self.fading[:]`. The `[:]` makes a shallow copy, so the loop iterates the copy while removing from the original. This is the correct and explicit fix for the mutation-during-iteration hazard that `check_collision_ball()` only dodges by luck. The same author wrote both; one is right for the right reason.

Effects outlive the round on purpose

`main.py` line 132 calls `block.update_effects()` in the `else` branch too, so a burst that was in flight when the last life was lost keeps animating on the game-over screen instead of freezing mid-air. This is a small, deliberate touch and it is easy to miss when reading the loop.

6 paddle.py and scoreboard.py

6.1 The paddle

The simplest module, 34 lines. It binds Left and Right arrows to methods that step ± 20 units, and clamps itself to ± 320 each frame so it cannot leave the screen.

onkeypress without onkeyrelease means movement is not smooth

The paddle moves 20 units per key *event*, not per frame. Held-key repeat is handled by the operating system, so the paddle's speed and its initial delay before repeating are the OS keyboard settings, not the game's. The frame-based way to do this is to record the key as held on press, clear it on release, and move by a fixed amount each frame inside the loop. That would make paddle motion independent of keyboard settings, and it is a two-method change.

6.2 The scoreboard and persistence**Jargon: Persistence**

Keeping data after the program exits. Variables live in memory and die with the process; to survive, a value must be written to a file (or a database). Here the single persisted value is the high score, in `highestScore.txt`.

The read happens at **module level**, at lines 3 to 8, outside any function or class:

```
1 try:
2     score = int(open('highestScore.txt', 'r').read())
3 except FileNotFoundError:
4     score = open('highestScore.txt', 'w').write(str(0))
5 except ValueError:
6     score = 0
```

Verified bug: a first run starts the high score at 1, not 0

`file.write()` returns **the number of characters it wrote**, not the value written. Writing the string "0" writes one character, so `write()` returns 1, and the `FileNotFoundError` branch assigns `score = 1`.

On any machine with no `highestScore.txt`, the game therefore opens claiming a high score of 1. The player must destroy two blocks before their score is recognised as beating it. The README states this branch means the game “just starts the high score at 0”. It does not. Confirmed by running the module with the file absent:

```
1 H1 module-level `score` on first run (no file) = 1
```

The fix is one line: assign `score = 0` and write the file as a separate statement. The bug is invisible after the first run, which is exactly why it survived.

Import has a side effect, and it is a write

Reading the file at module level means `import scoreboard` touches the disk, and in the missing-file case *creates* a file. An import should not do that: it makes the module impossible to test without a filesystem, impossible to reuse with a different save location, and it means the read happens once per process no matter how many `Scoreboard` objects exist. The README's “What I'd do differently” identifies this correctly and it is the right call.

There is a second, quieter consequence. Because the file is read once at import, the value in `Scoreboard.highScore` is a snapshot from process start. Two copies of the game running at once would each overwrite the other's high score with a stale value on exit.

increase_score increments the high score instead of assigning it

```

1 self.score += 1
2 if self.score > self.highScore:
3     self.highScore += 1

```

It happens to be correct, because `score` only ever rises by 1, so the moment it overtakes `highScore` the two are equal and stay locked together. But the intent is “the high score is the best score”, and the code says “add one to the high score”. `self.highScore = self.score` expresses the intent and would survive any future change that awarded more than one point per block. As written, awarding 5 points per block would silently break the high score.

Scoreboard.reset() is dead code

Lines 56 to 58 define `reset()`, which calls `save_high_score()` then `new_round()`. Nothing calls it: `main.py` calls `save_high_score()` and `new_round()` separately, at the two moments it actually wants each. Confirmed by `grep` across the project. It is a leftover from an earlier structure and should be deleted.

7 Honest findings

The boxes above cover each issue where it arises. This section collects the ones found by **running** the code rather than reading it, since those are the ones an interviewer could surface that reading alone would not.

7.1 The ghost block

Verified bug: an invisible block sits at the origin and scores a free point

Trace `make_block()` carefully:

```

1 def make_block(self):
2     block = Turtle()                                # a new turtle is born at (0, 0)
3     ...
4     self.place_block(block)                         # may return early WITHOUT calling goto()
5     row = int(round((UPPER_YCOR - block.ycor()) / 20))
6     ...
7     self.all_blocks.append(block)                   # appended REGARDLESS

```

When the wall runs out of vertical room, `place_block()` sets `can_make_more = False` and returns *before* calling `block.goto()`. The block is left at the turtle default position, (0,0), the dead centre of the play field. `make_block()` then appends it to `all_blocks` anyway. `make_all_blocks()` calls `hideturtle()` on it, which makes it invisible but does **not** remove it from the list.

The consequences are all real:

- `check_collision_ball()` iterates `all_blocks`, so the ghost is a live collider. The ball starts at (0,0): distance 0. On the **first frame of every game**, the player scores a free point off an invisible block, `dy` flips so the ball launches *downward* rather than upward, and a particle burst fires at the centre of the screen from nowhere.
- `len(block.all_blocks) == 0` is the win condition, so the win requires destroying the ghost too. It is destroyed on frame one, so the win still works, but by luck.
- The row calculation for the ghost is `round((240 - 0) / 20) = 12`, and `ROW_COLORS[12 % 7]` is 'steel blue'. The burst at the centre is steel blue, a colour no real row ever uses.

Confirmed by running the real `Block` and `Ball` classes:

```
1 H2 len(all_blocks) after make_all_blocks = 50
2 H2 last block pos = (0.00,0.00) isvisible = False color = steel blue
3 H4 ball start pos = (0.00,0.00) score before = 0
4 H4 collision on first frame = True score after = 1 ball.dy = -1.05
```

Every visible symptom of this bug (the centre burst, the ball starting downward, the score reading 1) looks like a feature if you have never read the code. The fix is one line: do not append the block when `can_make_more` went false, or `pop()` it in `make_all_blocks()` instead of merely hiding it.

7.2 The dead bottom wall

Verified dead code: the bottom wall bounce can never execute

`ball.py` bounces off the bottom wall at `ycor() < -290`. `check_game_over()` fires at `ycor() < -280`. In the loop (Figure 3), the ball moves, then the wall is checked, then, at the end of the same iteration, game-over is checked. The ball’s maximum speed is 2.6 units per frame, so it cannot cross the 10-unit gap between `-280` and `-290` within one frame. Game-over always fires first, and the ball is teleported away.

Driving the ball straight down at maximum speed through main’s exact per-frame order:

```
1 game_over fired at y = -280.6 on frame 30
2 lowest y ever reached before game_over = -280.6
3 bottom-wall bounce threshold is y < -290 ; reachable? False
```

The bottom wall is unreachable code. This is harmless (it is a floor under a floor) but it is misleading: the README claims “the ball bounces off all four walls”, and it bounces off three. The code reads as though there is a safety net that does not exist.

7.3 Two unreachable colours

ROW_COLORS has 7 entries and the wall can only ever have 5 rows

Rows are placed at `y = 240, 220, 200, 180, 160`; the next would be 140, which fails the `y > LOWER_YCOR` test since `LOWER_YCOR = 140`. So there are always exactly five rows, using `ROW_COLORS[0..4]`. ‘steel blue’ appears only on the invisible ghost block, and ‘medium purple’ never appears at all.

The comment on line 10, “cycled if there are more rows than colors”, describes a case the geometry makes impossible. The modulo in `ROW_COLORS[row % len(ROW_COLORS)]` is defensive code guarding against nothing. Confirmed:

```
1 H3 distinct y rows present = [0, 160, 180, 200, 220, 240]
2 H3 colors in use = ['gold', 'medium sea green', 'orange', 'steel blue',
3                   'tomato', 'yellow green']
```

(The 0 row and ‘steel blue’ in that output are the ghost block.)

7.4 Documentation drift

Three README claims the code contradicts

1. **“a 279-line turtle game”** (“What I’d do differently”). Measured: `wc -l` gives 475 lines across the five modules ($134 + 147 + 102 + 58 + 34$). The figure is stale by roughly 40%.
2. **“uses only ...turtle ...and time”** (“Setup”). Nothing imports `time`. The actual imports are `turtle`, `random` and `math`.
3. **“a corrupted file doesn’t crash the game, it just starts the high score at 0”** (“Challenges”). It starts at 0 for a corrupted file (the `ValueError` branch) but at **1** for a missing file, as proven above.

These are cheap to fix and expensive to be caught on.

7.5 Verification note

What was actually run, and what was not

Every claim in this section was checked by importing the project’s real modules against a real display and driving them, working on copies in a scratch directory so the repository’s own `highestScore.txt` was never read or written. The probes replicate `main.py`’s exact per-frame call order.

The full interactive game was **not** played end to end: `main.py` never returns until the window closes, and neither the paddle’s key bindings nor the visual appearance of the particle bursts were exercised. Claims about how things *look* (the flash, the burst, the colour bands) are read from the code and are marked as such rather than seen.

8 What the project gets right

The honest findings are numerous, so it is worth being equally precise about the parts that are genuinely well done, because they are the parts worth defending:

- **The file split earns itself.** Each collision rule is a different shape of problem and each lives in the module that owns the relevant object. This is the right decomposition and it is the reason the bugs above are findable at all.
- **`tracer(0)` plus a manual `update()`** is the correct and non-obvious way to get frame-based rendering out of `turtle`.
- **The two-reset design** (keep difficulty after a miss, drop it on restart) is a real game-feel decision, not an accident.
- **Clamp-then-flip** in the wall and paddle collisions shows the author hit the stuck-vibrating bug and fixed it properly.
- **The `[:]` copy** in `update_effects()` shows awareness of the mutation-during-iteration hazard.
- **Catching Terminator** so closing the window exits cleanly rather than dumping a traceback is a detail most projects at this size skip.
- **The README’s “What I’d do differently” is largely correct**, and identifies the module-level file read, the distance check and the double collision call unprompted. The drift documented above is in the details, not the judgement.

The honest summary

This is a competent exercise that does the thing it set out to do: it builds a working game loop, a state machine and three kinds of collision detection without an engine, and the author can explain why each file exists. It also ships a free point from an invisible block on frame one, a high score that starts at 1, a wall of three walls, and a paddle that cannot be aimed with. Every one of those is a small fix and a good story about how it was found.